Legends of Sword and Wand

Software Design Project Deliverable One

Jaelan Cruz, Tobi Kasumu
Marios Fokaefs

CSSD 2203

Due March 25th @11:59 PM

Document Change Control

Version	Date	Author(s)	Summary of Changes
0.1	Feb. 21	Jaelan	Initial draft, title, table of contents, introduction
0.2	Feb. 23	Jaelan, Tobi	Sequence Diagrams
0.3	Feb. 28	Tobi	Major Design Decisions, Use of Design Patterns, Activities Plan
0.4	Mar. 01	Jaelan, Tobi	Architecture, Class Diagrams, Test Cases, AI notations
0.5	Mar. 02	Jaelan	Revising
0.6	Mar.25	Tobi	Date Fiex
0.7	Mar. 25	Jaelan	Test Case Fixes: - Reduced test cases from 27 to 12, as according to instructions - Related test cases closer to use cases

Table of Contents

1. Introduction

2. Sequence Diagrams

3. Major Design Decisions

4. Architecture

5. Class Diagrams

6. Use of Design Patterns

7. Activities Plan

8. Test Driven Development

9. Use of AI

—

1. Introduction

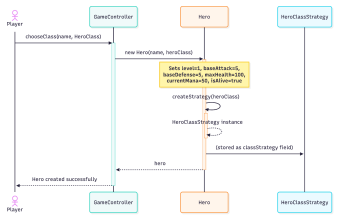
Purpose

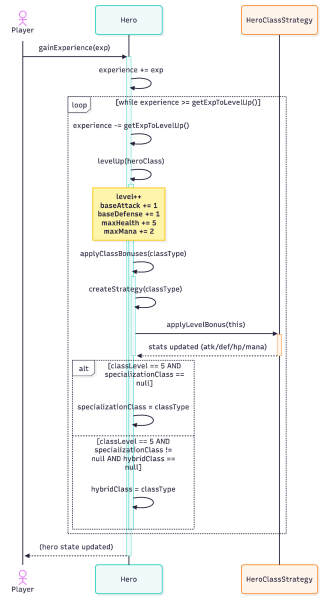
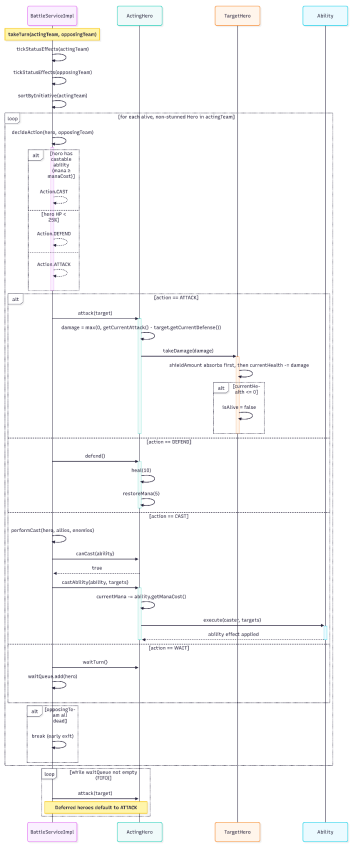
Legends of Sword and Wand is a Java-based RPG system with inspiration from Dungeons & Dragons. This document will go over the mind behind the project, including design decisions, class structure and architecture, and design patterns. The learning outcome of this project is to not just learn to code, but to apply theoretical architecture by creating an intrinsic system with further in-depth knowledge of object oriented programming.

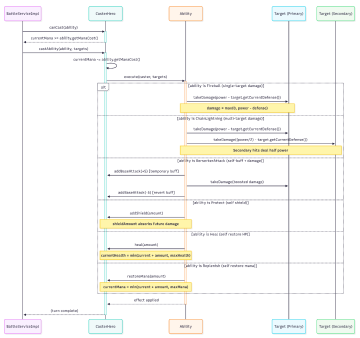
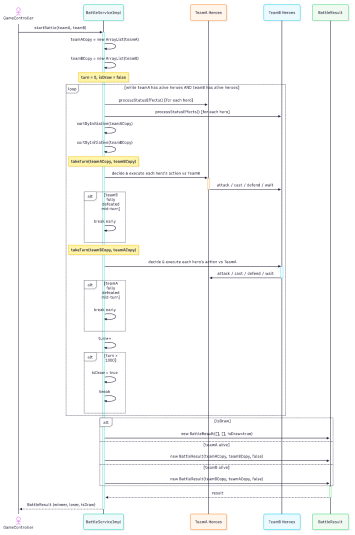
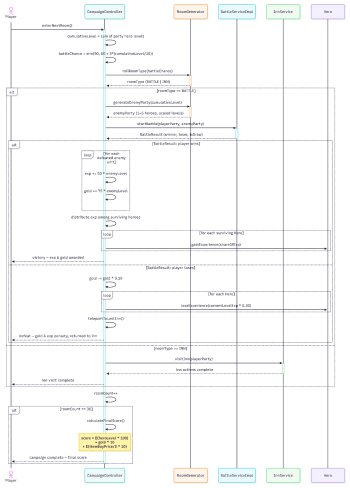
Overview

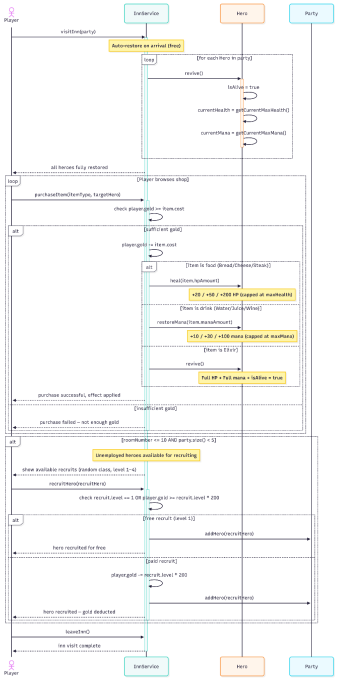
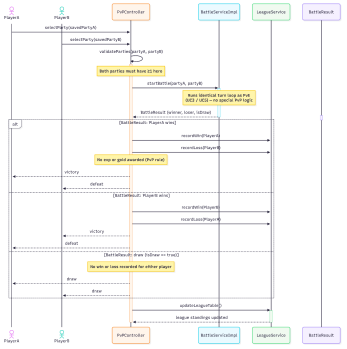
Just as was learned in CSSD 2101, the system architecture is composed of layers: domain/model layer, service layer, persistence, and testing layer. This allows other collaborators to easily navigate throughout the codebase and work without conflict. As for the actual RPG, the system includes four hero classes (Order, Chaos, Warrior, Mage) with specialization and hybridization progression, in which those heroes engage in turn-based combat. The core battle engine is deterministic (apart from Chain Lightning target shuffling), and all class-specific behavior is encapsulated behind the Strategy pattern instead of hardcoding conditions. The RPG will feature two gamemodes, PvP (player vs player) and PvE (player vs environment).

2. Sequence Diagrams

#	Use Case	UML
UC1 - Hero Creation	Creation/recruiting of a new hero, providing a name and class, while the system instantiates a new hero object, applying stats based on the class selected via HeroClassStrategy.java (strategy design pattern)	

UC2 Hero-Level-Up	Level up function of a hero, applying base stat growth when experience threshold is passed, then performs a level check to trigger specialization, and if the second class is also level 5, then hybridization is set too.	
UC3 Execute-Battle-Turn	A sequence of events that sorts heroes by initiative, also ticking any status effects. For every unit that is not stunned and alive, they can choose one of four actions: atk, def, cast, or wait.	

UC4 Cast-Ability	Verifies that the hero has enough mana to cast an ability, then deducts the mana cost and executes the ability cast, applying tick damage to specified targets.	
UC5 Full-Battle	This use case will encompass the entire lifecycle of a team vs team battle, creating a copy of both teams so that the original teams are not modified, then entering the main battle loop.	
UC6 PvE-Campaign-Room	A sequence of events when the player enters a room during a PvE campaign. The probabilities of an encounter are dependent on the cumulative level of the player's party.	

UC7 Inn-Visit	Descriptive overview of what happens when the party enters an inn, which occurs on a roll or after defeat. All fallen heroes are revived at the inn and the player can browse through the shop to purchase items. Heroes may also be available for recruitment at the cost of money depending on the level of the hero.	
UC8 Pvp-Battle	Players can fight against each other using a saved party of their choice from a PvE campaign, though no experience or gold is awarded from PvP.	

3. Major Design Decisions

Separation of Concerns:

- All business rules (damage, leveling, specialization, status effects) are implemented inside domain classes such as Hero, Ability, and StatusEffect.
- BattleServiceImpl only orchestrates the battle flow. This improves testability and maintainability.

Strategy Pattern for Hero Classes:

- Each hero class (Order, Chaos, Warrior, Mage) has its own HeroClassStrategy implementation.
- This removes conditional logic from Hero and makes the system extensible.

Template Method for Abilities:

- Ability defines the fixed casting process (mana check → deduct mana → execute).
- Each ability only implements execute(). This ensures consistent casting behavior.

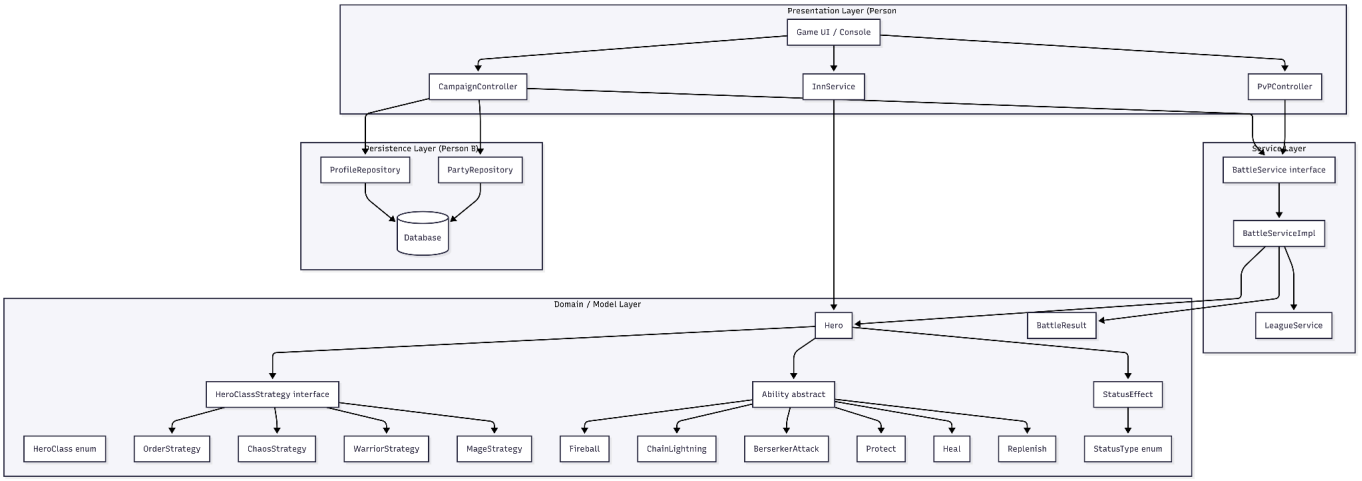
Factory Method for Strategy Creation:
- Hero.createStrategy() instantiates the correct strategy based on HeroClass.
- This centralizes object creation and prevents coupling.

Enum as Identity Only:
- HeroClass is used only as an identifier. All behavior exists in strategy classes.

Deterministic Battle Engine:
- Damage uses max(0, atk - def).
- Turn order and status effects follow a predictable flow, allowing reliable testing.

4. Architecture

System Architecture



Module Table

Module	Description	Exposed Interface
M1: Domain Model	Core game entities: Hero, Ability, HeroClassStrategy, StatusEffect, BattleResult, enums. Contains all business rules.	Hero public API, Ability.execute(), HeroClassStrategy interface

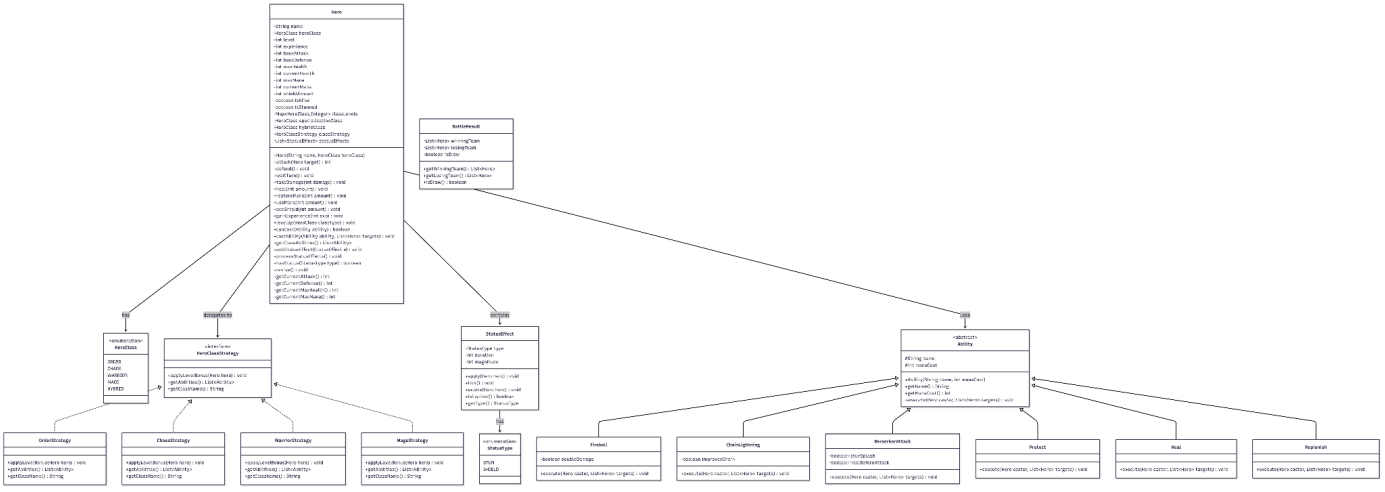
M2: Battle Service	Orchestrates turn-based battles between teams. Stateless.	BattleService.startBattle(List<Hero>, List<Hero>) : BattleResult
M3: PvE Campaign (Person B)	Campaign room loop, enemy generation, exp/gold distribution, score calculation.	CampaignController.enterNextRoom( )
M4: Inn Service (Person B)	Hero revival, item shop, hero recruitment.	InnService.visitInn(List<Hero>)
M5: PvP Controller (Person B)	Party selection, PvP battle dispatch, league recording.	PvPController.startPvP(partyA, partyB)
M6: Persistence (Person B)	Database read/write for hero profiles, party state, campaign progress, league table.	ProfileRepository, PartyRepository

Interface Operations

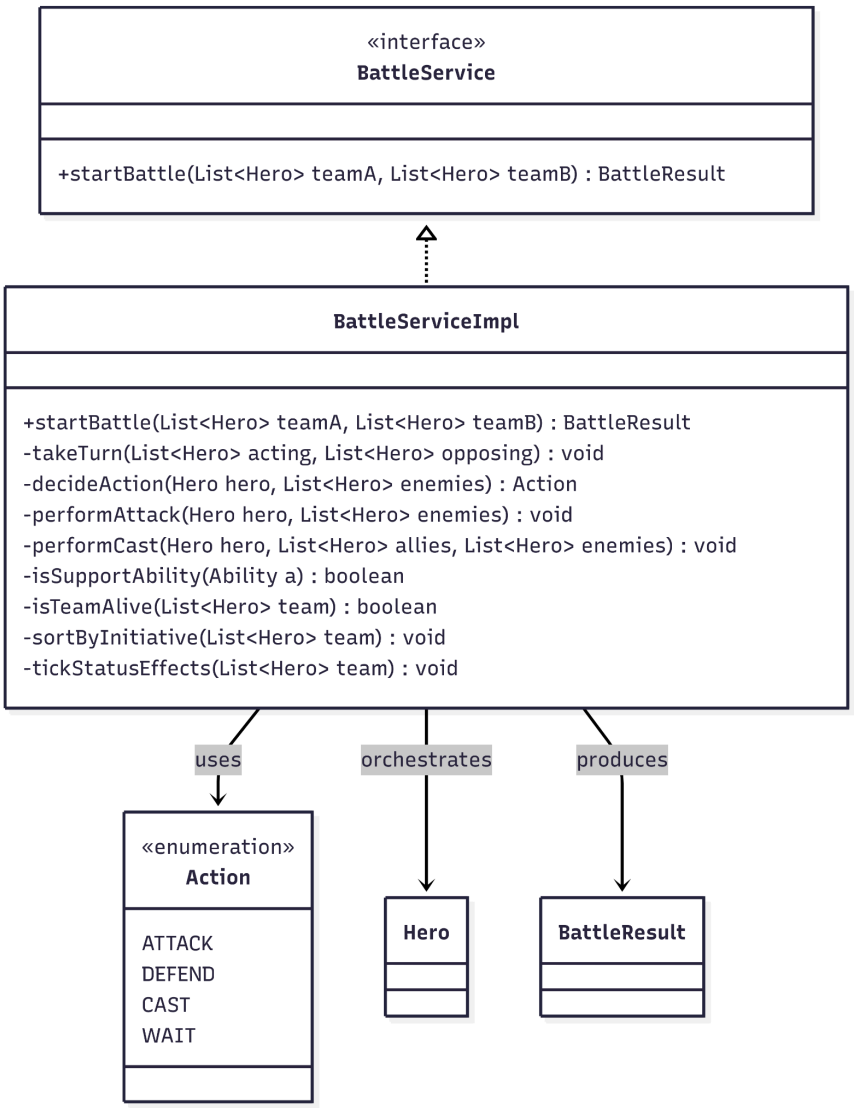
Interface	**Operations**
BattleService	BattleResult startBattle(List<Hero> teamA, List<Hero> teamB) — used by M3, M5
HeroClassStrategy	void applyLevelBonus(Hero hero) — used by M1 Hero.levelUp() List<Ability> getAbilities() — used by M1 Hero.getClassAbilities()
Ability	void execute(Hero caster, List<Hero> targets) — used by M1 Hero.castAbility()

5. Class Diagrams

Core Domain Model (Covers Heroes, Classes, Ability, etc.)



Battle Service (Covers the battle interface, battle implementations, actions)



Class Summary Table

Class	Role
Hero	Central domain object. Holds stats, class levels, status effects, strategy.
HeroClass	Enum — identity key for the four classes and HYBRID.
HeroClassStrategy	Strategy interface — separates class behaviour from class identity.
OrderStrategy	Concrete strategy: +5 mana/+2 def per level; abilities: Protect, Heal.
ChaosStrategy	Concrete strategy: +3 atk/+5 hp per level; abilities: Fireball, ChainLightning.

WarriorStrategy	Concrete strategy: +2 atk/+3 def per level; ability: BerserkerAttack.
MageStrategy	Concrete strategy: +5 mana/+1 atk per level; ability: Replenish.
Ability	Abstract base for all abilities. Holds name and mana cost.
Fireball	Damages up to 3 enemies. Sorcerer variant doubles damage.
ChainLightning	Damages all enemies with decay (25% carry-over; 50% for Invoker).
BerserkerAttack	Primary + 2 splash at 25%. Knight variant stuns; Paladin heals caster first.
Protect	Shields all allies for 10% of each hero's max HP.
Heal	Heals the ally with the lowest current HP for 25% of their max HP.
Replenish	Restores 30 mana to all allies, 60 to caster.
StatusEffect	Encapsulates a timed status (STUN, SHIELD) with apply/tick/expire logic.
StatusType	Enum — STUN, SHIELD.
BattleResult	Value object — winning team, losing team, isDraw flag.
BattleService	Interface defining the battle contract.
BattleServiceImpl	Orchestrates the full battle loop; all business rules stay in domain classes.

6. Use of Design Patterns

Strategy (Implemented):

- Encapsulates hero class behavior.
- Allows new classes without modifying existing code.

Template Method (Implemented):

- Controls the ability casting workflow.
- Ensures consistent mana validation and execution.

Factory Method (Implemented):

- Used to create hero strategies internally within Hero.

Singleton (Planned – D2):

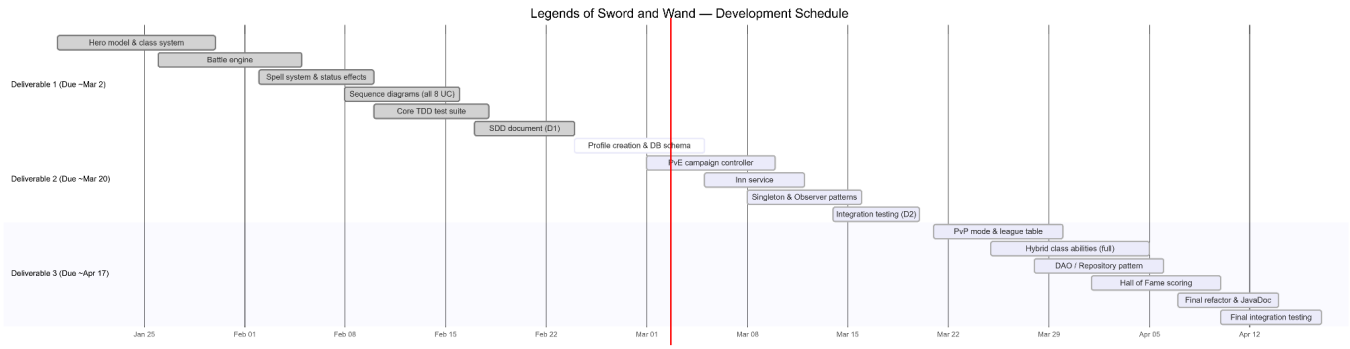
- Will be used for LeagueService and CampaignController to ensure one shared instance.

Observer (Planned – D2):
- Will handle HP changes and level-up events in a decoupled way.

Repository / DAO (Planned – D3):
- Will abstract database operations from domain logic.

7. Activities Plan

Gantt Diagram



Product Backlog

ID	Item	Deliverable
PB-01	Hero model + class system	D1 (done)
PB-02	Battle engine (turn-based)	D1 (done)
PB-03	Spell / ability system	D1 (done)
PB-04	Status effects (stun, shield)	D1 (done)
PB-05	Sequence diagrams (all 8 UC)	D1 (done)
PB-06	Core test suite (27 tests)	D1 (done)
PB-07	Profile creation & persistence	D2
PB-08	PvE campaign controller	D2

PB-09	Inn service	D2
PB-10	Database integration (SQLite)	D2
PB-11	Singleton pattern (LeagueService, CampaignController)	D2
PB-12	Observer pattern (HP/level-up events)	D2
PB-13	PvP mode + league table	D3
PB-14	Hybrid class ability upgrades (full set)	D3
PB-15	Repository / DAO pattern	D3
PB-16	Hall of Fame scoring	D3
PB-17	Full integration testing	D3
PB-18	Final refactoring & JavaDoc	D3

8. Test Driven Development

All 12 use case tests pass as of Deliverable 2

Test ID	Category	Requirements Coverage	Initial Condition	Procedure	Expected Outcome	Notes
TC-01	Profile Creation	UC1 - Profile creation & persistence	System initialized; no profile named `P1` exists	1. Call `profileService.createProfile("P1")`.	Profile `P1` is created and saved; initial gold is 0.	Uses in-memory repository for deterministic result.
TC-02	Hero Level-Up	UC2 - Hero level-up/s	Profile `P1` exists	1. Create/add `H1` to	`H1` level increases by 1 and	Mirrors use-case

		pecialization trigger check	with hero `H1` class `ORDER` at level 1	party. 2. Call `H1.levelUp(ORDER)`.	class growth is applied.	test `UCT-02`.
TC-03	Battle Turn Action	UC3 - Execute battle turn	Two heroes alive on opposite teams	1. Start battle with one stronger and one weaker hero. 2. Let one turn execute via battle engine.	Turn resolves correctly and health/status updates are applied by combat rules.	Covered through full battle execution path.
TC-04	Ability Casting	UC4 - Cast ability	Caster has enough mana and valid targets exist	1. Cast `Protect` (or `Replenish`) through ability flow.	Mana is deducted and ability effect is applied to targets.	Ensures cast pipeline: check mana -> deduct -> execute.
TC-05	Full Battle Outcome	UC5 - Full battle lifecycle	Team A stronger than Team B	1. Call `battleService.startBattle(teamA, teamB)`.	Battle completes with Team A as winner; result is not draw.	Validates end-to-end battle flow.
TC-06	Campaign Progression	UC6 - PvE campaign room flow	Profile has at least one active hero	1. Call `campaignService.startCampaign(profile)`. 2. Call `enterNextRoom(profile)`.	Campaign becomes active and room index increments from 0 to 1.	RNG is seeded for deterministic behavior in tests.

TC-07	Inn Shop Purchase	UC7 - Inn visit/shop	Profile has hero and sufficient gold (`>= item cost`)	1. Purchase `InnItem.BREAD` for hero.	Gold decreases by item cost; HP effect applied and capped at max HP.	Matches use-case test `UCT-04` edge-case fix.
TC-08	Inn Recruitment	UC7 - Inn recruitment	Party has available slot; candidate hero available	1. Recruit level-1 candidate.	Recruitment succeeds and candidate is added to party.	Level-1 recruit is free by design.
TC-09	Inn Revive	UC7 - Inn revive behavior	Party contains a dead hero (`HP == 0`)	1. Call `innService.visitInn(profile)`.	Dead hero is revived with full HP and mana.	Confirms revive/reset behavior.
TC-10	Draw Condition	UC5 - Full battle edge case	Two highly defensive heroes	1. Run battle to completion.	Battle ends as draw when turn limit is reached.	Verifies loop termination rule.
TC-11	Party Size Guard	UC1 - Hero/party management	Active party already contains 5 heroes	1. Attempt to add a 6th hero.	Add request fails (`null` or rejected), size remains 5.	Prevents invalid party growth.
TC-12	PvP/League Recording	UC8 - PvP ranking persistence	Fresh league repository and singleton reset	1. Record result `A` beats `B` in league service.	`A` gains one win and `B` gains one loss in table.	Validates singleton + persistence interaction.

8.1 Test Traceability Matrix

This is to help with tracing each use case according to test cases:

Use Case	**Test IDs**	**Use-case Tests (`src/test/java/service/UseCaseTests.java`)**
UC1 - Hero Creation / Party Management	TC-01, TC-11	UCT-01, UCT-02, UCT-03
UC2 - Hero Level-Up	TC-02	UCT-02
UC3 - Execute Battle Turn	TC-03	UCT-08, UCT-09
UC4 - Cast Ability	TC-04	UCT-10, UCT-11
UC5 - Full Battle	TC-05, TC-10	UCT-08, UCT-09
UC6 - PvE Campaign Room	TC-06	UCT-06, UCT-07
UC7 - Inn Visit	TC-07, TC-08, TC-09	UCT-03, UCT-04, UCT-05
UC8 - PvP Battle / League Record	TC-12	UCT-12

9. Use of AI

GitHub Copilot (GPT-4o) was used to assist with this project. The majority of use cases involved code supervision, planning, and refactoring. More specifically ensuring code integrity and architecture, same way as a senior developer would oversee an operation.
GPT-4o was also used in assisting documentation implementation, as well as UML diagram support. GPT-4o was used to review UML diagrams, making sure they're correct and concise, and implementing the specificities of classes for extensive detailing.
GPT-5 mini was recently used to oversee test case implementation operations, refining and revising wherever necessary.